

Agora — Project Study Guide

Open in VS Code and press **Ctrl + Shift + V** to see this as a formatted document.
Every section follows the actual execution order of the app.

Table of Contents

1. [Tech Stack](#)
 2. [App Startup](#)
 3. [Database Models](#)
 4. [Authentication](#)
 5. [Campus Core](#)
 6. [Campus Features](#)
 7. [Chat Rooms](#)
 8. [Study Rooms](#)
 9. [Campus Map & Events](#)
 10. [Frontend Architecture](#)
 11. [Key Questions to Prepare](#)
-

1. Tech Stack

Layer	Technology	What it does
Language	Python 3	All backend logic
Web framework	Flask	Handles HTTP requests and routing
Database ORM	SQLAlchemy (via Flask-SQLAlchemy)	Talks to the database using Python objects instead of raw SQL
Database	SQLite (local) / PostgreSQL (deployed)	Stores all data
Auth sessions	Flask-Login	Keeps users logged in across requests
Email	Flask-Mail	Sends verification and password reset emails
Real-time chat	Flask-SocketIO	WebSocket connection for study room messages
Password hashing	bcrypt	Securely stores passwords (never in plain text)
Signed tokens	itsdangerous	Generates tamper-proof email verification tokens
Templates	Jinja2	HTML templates with Python variables injected
Frontend	Vanilla JS + CSS	No framework — plain JavaScript
Map	Leaflet.js	Interactive campus map with pins

2. App Startup

`index.py` — The Entry Point

```
from app import create_app
app = get_application()

if __name__ == '__main__':
    socketio.run(app, debug=True)
```

- This is the **first file Python runs**.
- It calls `create_app()` from `app/__init__.py` and stores the result in `app`.

- If startup fails (e.g., a missing config), it catches the error and returns a **fallback Flask app** that shows the error page instead of crashing silently. This is important for deployment.
- `socketio.run(app)` is used instead of `app.run()` because SocketIO needs to control the server loop.

`app/extensions.py` — Extension Instances

```
db          = SQLAlchemy()      # database
login_manager = LoginManager() # session management
socketio     = SocketIO()       # real-time WebSocket
mail         = Mail()           # email sending
```

- These are created **without an app** attached yet.
- Why? Because of the **Application Factory Pattern** — you want to create extensions once, then attach them to different app instances (e.g., one for testing, one for production).
- `login_manager.login_view = 'auth.login'` tells Flask-Login which route to redirect to when a user tries to access a `@login_required` page without being logged in.

`app/__init__.py` — The App Factory

```
def create_app(config_class=Config):
    app = Flask(__name__)
    app.config.from_object(config_class)
```

This function **builds and returns the Flask app**. Called once at startup. Key things it does:

Database selection

```
is_deployed = os.environ.get('VERCEL') or os.environ.get('RAILWAY_ENVIRONMENT') ...
if DATABASE_URL and is_deployed:
    # use PostgreSQL
else:
    # use SQLite locally at instance/campus.db
```

- Locally: uses **SQLite** (a single file, no server needed).
- On deployment (Railway/Vercel): uses **PostgreSQL** (a proper server database).

Initializing extensions

```
db.init_app(app)
login_manager.init_app(app)
mail.init_app(app)
socketio.init_app(app, cors_allowed_origins='*', async_mode=...)
```

- This is where the extensions created in `extensions.py` get "attached" to the app.

Registering blueprints

```
app.register_blueprint(auth_bp)    # /auth/...
app.register_blueprint(main_bp)   # /, /dashboard, /campus/...
app.register_blueprint(map_bp)    # /campus/.../map, /api/...
app.register_blueprint(study_bp)  # /campus/.../study-rooms/...
```

- A **blueprint** is a group of related routes. It keeps the code organized by feature instead of dumping all routes in one file.

Context processor — sidebar data

```
@app.context_processor
def inject_global_data():
    ...
    return {'sidebar_chat': ..., 'sidebar_study': ..., 'unread_notifications_count': ...}
```

- This function runs **before every page render** and injects variables into every template automatically.
- It populates the sidebar with the user's chat rooms, study rooms, and unread notification count.
- It skips JSON API endpoints (`req.is_json`) to avoid doing unnecessary database queries on every fetch call.

Creating tables

```
with app.app_context():
    db.create_all()
```

- On first run, creates all database tables if they don't exist yet.

3. Database Models

File: `app/models.py`

All models inherit from `db.Model`. Each class = one database table. Each attribute = one column.

User

```
id, name, email, password_hash, is_verified
```

- `password_hash`: password is **never stored in plain text** — bcrypt hashes it.
- `is_verified`: the user can't log in until they click the verification link in their email.
- `has_verified_domain(domain)`: checks if the user owns a verified email on a given domain (e.g., `usal.edu.lb`). Used for campus domain restrictions.
- `is_campus_owner(campus_id)` / `is_campus_admin(campus_id)`: helper methods used throughout routes to check permissions.

Campus

```
id, name, description, banner_image, map_image, creator_id, domain, center_lat, center_lng
```

- `creator_id` → foreign key to `User`
- `domain`: if set, only users with a verified email ending in this domain can join.
- Has relationships to `clubs`, `offices`, `members`, `chat_rooms`, `study_rooms`, `events`, `resources`.

CampusMember

```
user_id, campus_id, role ('owner' | 'admin' | 'user'), joined_at
```

- The **join table** between `User` and `Campus`.
- `UniqueConstraint('user_id', 'campus_id')` prevents a user from joining the same campus twice.

UserEmail

```
user_id, email, is_verified
```

- A user can have **secondary emails** (e.g., a university email different from their login email).

- Used to meet campus domain restrictions even if the primary email doesn't match.

Notification

```
user_id, title, message, is_read, link, created_at
```

- Created programmatically (when someone joins your campus, joins your study room, etc.).

Resource + ResourceRequest

- **Resource** : a file (PDF, DOCX, etc.) uploaded to a campus.
- **ResourceRequest** : a user asking for notes for a specific course/chapter.

TutorPost

```
full_name, email, role ('tutor'|'student'), courses, method
```

- The **Campus Market** — users post as tutors or students looking for help.

ChatRoom → ChatMember → ChatMessage

- **ChatRoom** : a named group chat inside a campus.
- **ChatMember** : which users are in which room. Tracks **last_read_at** for unread indicators.
- **ChatMessage** : individual messages. Uses **polling** (the frontend asks the server every second for new messages).

StudyRoom → StudyRoomMember → StudyRoomMessage

- Same structure as chat, but for time-limited study sessions.
- Uses **WebSockets (Socket.IO)** for real-time messaging instead of polling.
- **is_expired** property: a study room expires 2 hours after its **session_time**.

Event + EventParticipation + EventCreationLog

- **Event** : placed on the campus map with coordinates (**lat** , **lng**), a start time and end time.
- **EventParticipation** : tracks whether a user is "interested" or subscribed for a notification when the event starts.
- **EventCreationLog** : prevents regular students from creating more than one event every 4 days (admins bypass this).

Club + Office

- Buildings/locations pinned on the campus map by admins.

4. Authentication

File: `app/auth/routes.py`

Blueprint prefix: `/auth`

Templates: `auth/login.html`

JS/CSS: `auth.js` , `auth.css`

Email validation regex

```
EMAIL_RE = re.compile(r'^[^\s@]+@[^\s@]+\.(edu(\.[a-z]{2,})?|ac\.[a-z]{2,})$')
```

Only `.edu` or `.ac.xx` emails are accepted — enforces that Agora is for university students.

Register — `POST /auth/register`

1. Validates name, email (regex), password length.
2. Checks if email already exists (as a primary or secondary verified email).
3. If email exists but is unverified → resends the verification email instead of creating a duplicate.
4. Hashes the password with **bcrypt**: `bcrypt.hashpw(password.encode(), bcrypt.gensalt())` .
5. Creates the `User` with `is_verified=False` .
6. Sends a **verification email** with a signed token (via `itsdangerous`).

Email Verification Token — `send_verification_email()`

```
s = URLSafeTimedSerializer(SECRET_KEY)
token = s.dumps(email, salt='email-verify')
link = url_for('auth.verify_token', token=token, _external=True)
```

- `URLSafeTimedSerializer` creates a **cryptographically signed, time-limited token**.
- The token encodes the email address and is signed with `SECRET_KEY` — if anyone tampers with it, verification fails.

- Expires in **1 hour** (`max_age=3600`).

Verify Token — `GET /auth/verify/<token>`

```
email = s.loads(token, salt='email-verify', max_age=3600)
user.is_verified = True
login_user(user)
```

- Decodes the token, marks the user as verified, and logs them in.

Login — `POST /auth/login`

1. Validates email format.
2. Queries `User` by email.
3. Checks password: `bcrypt.checkpw(password.encode(), user.password_hash.encode())` .
4. Checks `is_verified` — unverified users cannot log in.
5. Calls `login_user(user)` — Flask-Login creates a session cookie.

Forgot Password Flow

1. User submits email → `POST /auth/forgot-password`
2. If user found, a **reset token** is generated: `s.dumps(user.id, salt='password-reset')` .
3. Reset link emailed: `GET /auth/reset-password/<token>` .
4. User submits new password → token decoded, password re-hashed, saved.
5. **Privacy**: the success message is always shown regardless of whether the email exists, to prevent user enumeration.

Secondary Emails — `POST /auth/emails`

- Users can add extra university emails (e.g., if their login email is Gmail but the campus requires `.edu`).
- Same verification flow as primary email.
- `has_verified_domain()` checks both primary and secondary emails.

Delete Account — `POST /auth/delete-account`

1. Blocks deletion if the user owns any campuses (they must delete those first).
2. Deletes uploaded files from disk.
3. Logs out, deletes the `User` record (cascade deletes all related data).

5. Campus Core

File: `app/main/routes.py` (first half)**Blueprint:** `main_bp` (no prefix)**Index —** `GET /`

- If logged in: fetches today's events from all the user's campuses and shows them on the homepage.
- If not logged in: shows the landing page.

Dashboard — `GET /dashboard`

```

created    = Campus.query.filter_by(creator_id=current_user.id).all()
memberships = CampusMember.query.filter_by(user_id=current_user.id).all()
joined     = [m.campus for m in memberships]
campuses   = created + [c for c in joined if c not in created]

```

- Shows all campuses the user created or joined.
- Template: `main/dashboard.html`

Create Campus — `GET/POST /create-campus`

1. Validates campus name is unique.
2. If a `domain` is provided, checks the creator has a verified email on that domain.
3. Saves uploaded banner and map images to `static/uploads/`.
4. Creates `Campus` record.
5. Auto-creates a `CampusMember` record with `role='owner'`.

Join Campus — `GET /join-campus` + `POST /campuses/<id>/join`

- The list page sorts campuses by member count (most popular first).

- When joining:
- Checks if already a member.
- Checks domain restriction: if `campus.domain` is set, user must have a verified email ending in that domain.
- Creates `CampusMember(role='user')`.
- Sends a **notification** to the campus creator.

Campus Settings — `GET/POST /campus/<id>/settings`

- Only accessible by the campus **owner** (`is_campus_owner()`).
- Handles: name, description, domain, banner/map image uploads.
- **Add Moderator**: finds a user by email, checks they've already joined the campus, sets their `role='admin'`.
- **Demote**: sets `role='user'` back.
- **Delete Campus**: only the owner can do this. Cascades to delete all rooms, messages, resources, events.

6. Campus Features

File: `app/main/routes.py` (second half)

Templates: `main/campus_resources.html`, `main/campus_market.html`, `main/notifications.html`

Campus Resources — `GET /campus/<id>/resources`

- Shows a merged, time-sorted feed of `Resource` (uploads) and `ResourceRequest` (requests for notes).
- **Upload** (`POST /resources/upload`): saves file to `static/uploads/resources/` with a UUID filename to avoid collisions. Stores the original filename separately so downloads work correctly.
- **Download** (`GET /resources/<id>/download`): uses `send_from_directory` with `as_attachment=True` and the original filename.
- **Delete** (`DELETE /resources/<id>`): only the uploader can delete. Also removes the file from disk.
- JS file: `campus_resources.js` — handles modals, file upload form, chapter tags, event delegation for delete buttons.

Campus Market — `GET /campus/<id>/market`

- List of `TutorPost` records — tutors offering help or students seeking it.
- **Create post** (`POST /market/post`): validates name, email, role, at least one course, and method. Returns JSON — the JS adds the card to the page without a reload.

- **Delete** (`DELETE /market/post/<id>`): only the poster can delete.
- JS file: `campus_market.js` — handles the modal, choice buttons (tutor/student, online/in-person), course tags.

Notifications — `GET /notifications`

- Lists all `Notification` records for the current user, newest first.
- **Mark as read** (`POST /notifications/<id>/read`): sets `is_read=True` .
- **Clear all** (`POST /notifications/clear`): deletes all notifications.
- **API** (`GET /api/notifications/unread-count`): returns a JSON count, polled by `main.js` every 30 seconds to update the badge in the nav.

Auto Event Notifications — `before_app_request`

```
@main_bp.before_app_request
def check_event_notifications():
```

- Runs **before every request** (for authenticated users).
- Checks if any events the user subscribed to (`want_notification=True`) have started but haven't sent a notification yet.
- Creates a `Notification` and sets `notification_sent=True` so it only fires once.

7. Chat Rooms

File: `app/main/routes.py` (chat section)

Templates: `main/chat_browse.html` , `main/chat_room.html`

JS: `chat_browse.js` , `chat_room.js`

How chat messaging works (polling, not WebSocket)

The chat room uses **HTTP polling** — the browser asks the server for new messages every second:

```
Frontend → GET /campus/<id>/chat/<room_id>/poll?after=<last_message_id>
Server   → returns array of new messages
```

This is simpler than WebSockets but less efficient. Study rooms use WebSockets instead.

Routes

Route	What it does
GET /campus/<id>/chat	Lists all chat rooms in the campus
POST /campus/<id>/chat/create	Creates a new room (JSON)
POST /campus/<id>/chat/<room_id>/join	Join a room (max 14 rooms per user)
GET /campus/<id>/chat/<room_id>	Opens the chat room page with message history
POST /campus/<id>/chat/<room_id>/send	Sends a message (JSON)
GET /campus/<id>/chat/<room_id>/poll	Returns new messages since <code>after</code> ID (JSON)
POST /campus/<id>/chat/<room_id>/leave	Leave a room
POST /campus/<id>/chat/<room_id>/delete	Delete a room (admin/owner only)

Unread tracking

- `ChatMember.last_read_at` is updated every time the user opens or polls the room.
- The sidebar checks if the latest message's `sent_at` is after `last_read_at` to show the unread dot.

8. Study Rooms

Files: `app/study/routes.py` , `app/study/events.py`

Templates: `study/browse.html` , `study/room.html`

JS: `study_browse.js` , `study_room.js`

Key difference from chat

Study rooms use **Socket.IO (WebSockets)** — the server *pushes* messages to clients instantly instead of clients polling.

Routes (in `study/routes.py`)

Route	What it does
GET /campus/<id>/study-rooms	Browse study rooms
POST /campus/<id>/study-rooms/create	Create a room (JSON)
POST /campus/<id>/study-rooms/<id>/join	Join a room
POST /campus/<id>/study-rooms/<id>/leave	Leave (deletes room if no members left)
POST /campus/<id>/study-rooms/<id>/delete	Delete (owner only)
GET /campus/<id>/study-rooms/<id>	Open the study room chat page

Business rules

- You can only be in **one active study room per campus** at a time.
- You can't join a room that **overlaps in time** with another room you're in.
- Rooms older than 2 hours past their `session_time` are **auto-deleted** on browse.
- New rooms must be at most **1 week in the future**.

Socket.IO events (in `study/events.py`)

Client emits	Server does
<code>join_study_room</code>	Adds client to socket room, sends message history
<code>send_study_msg</code>	Saves message to DB, broadcasts to all in room
<code>leave_study_room</code>	Removes client from socket room

```
# Server broadcasts to everyone in the room EXCEPT sender:
emit('study_msg', {...}, to=_room_key(room_id), include_self=False)
# Then sends a "mine: True" version back to just the sender:
emit('study_msg', {..., 'mine': True})
```

This is how `mine: True/False` works — the same message is sent twice with different flags so each user sees their own messages on the right and others on the left.

9. Campus Map & Events

File: `app/map/routes.py`

Blueprint: `map_bp` (no prefix)

Template: `main/campus_map.html`

JS: `campus_map.js` , `map.js`

The map page

- Uses **Leaflet.js** — an open-source JavaScript mapping library.
- The base map layer is loaded from OpenStreetMap tiles.
- On load, JS calls `GET /api/map-data/<campus_id>` to fetch all pins (clubs, offices, events) as JSON, then places markers on the map.

Pin types

Type	Who can add	Stored in
Club	Admins only	<code>Club</code> table (with <code>lat</code> , <code>lng</code>)
Office	Admins only	<code>Office</code> table
Event	Anyone (students: 1 per 4 days)	<code>Event</code> table

Event creation cooldown

```
recent_event = EventCreationLog.query.filter(  
    EventCreationLog.user_id == current_user.id,  
    EventCreationLog.created_at >= four_days_ago  
)<div data-bbox="125 736 792 800" data-label="List-Group">

- EventCreationLog records when a student creates an event.
- If a log entry exists within the last 4 days, creation is blocked with a "try again in X" message.
- Admins skip this check entirely.

```

Event interest & notifications

- Each user can toggle "interested" and "notify me" on events.
- `EventParticipation` tracks both flags.

- `want_notification=True` → when the event starts, `check_event_notifications()` (in `main/routes.py`) creates an in-app `Notification`.
- `Route: POST /api/event/<id>/interest` — toggling "notify" also auto-sets "interested".

Campus center

- Admins can right-click on the map to set `campus.center_lat` / `campus.center_lng`.
- The map uses this as the starting view center when opened.

10. Frontend Architecture

Template inheritance — `base.html`

All pages extend `base.html`:

```
{% extends "base.html" %}
{% block content %}...{% endblock %}
{% block extra_css %}<link rel="stylesheet" href="...">{% endblock %}
{% block extra_js %}<script src="..."></script>{% endblock %}
```

- `base.html` provides the navbar, sidebar, theme toggle, and notification badge.
- Each page injects its own CSS and JS through blocks.

CSS files

File	Used by
<code>base-styles.css</code>	Every page (loaded in <code>base.html</code>)
<code>auth.css</code>	Login/register page only
<code>chat_room.css</code>	Chat room page
<code>study_room.css</code>	Study room chat page
<code>campus_resources.css</code>	Resources page
<code>campus_market.css</code>	Market page
<code>chat_browse.css</code>	Chat room list page
<code>campus_settings.css</code>	Campus settings page
<code>study_browse.css</code>	Study room list page

JS files

File	Purpose
<code>main.js</code>	Theme toggle, hamburger menu, nav active states, notification polling (every 30s)
<code>auth.js</code>	Tab switching, password strength meter, form validation
<code>chat_room.js</code>	Polling loop, send message, leave/delete overlay
<code>study_room.js</code>	Socket.IO connection, send/receive messages
<code>chat_browse.js</code>	Join rooms, create rooms
<code>study_browse.js</code>	Join/create study rooms
<code>campus_resources.js</code>	Upload modal, request modal, delete, chapter tags
<code>campus_market.js</code>	Post modal, choice buttons, course tags, delete
<code>campus_settings.js</code>	Live preview (banner/map/name/desc), delete overlay, revoke confirm
<code>campus_map.js</code> / <code>map.js</code>	Leaflet map, pin placement, event interest buttons

How Flask variables reach JavaScript

Since external `.js` files can't use Jinja `{{ }}` syntax, data is passed via `data-*` HTML attributes:

```
<!-- In the template -->
<div id="msgList" data-campus-id="{{ campus.id }}" data-room-id="{{ room.id }}">

<!-- In the JS file -->
const CAMPUS_ID = parseInt(document.getElementById('msgList').dataset.campusId);
```

Authentication state in JS (`main.js`)

```
<!-- base.html -->
<body data-authenticated="{{ 'true' if current_user.is_authenticated else 'false' }}">
```

```
// main.js
if (document.body.dataset.authenticated === 'true') {
    // start polling for notifications
}
```

Theme toggle

- A CSS class `dark` / `light` is toggled on `<html>`.
- The preference is saved to `localStorage` so it persists across page loads.
- CSS variables (`--base-bg` , `--base-text` , etc.) change based on the theme class.

11. Key Questions to Prepare

Q: What is Flask and why did you use it?

Flask is a Python web framework. It maps URLs to Python functions (routes) that return HTML responses. We used it because it's lightweight, easy to set up, and fits well with SQLAlchemy and SocketIO.

Q: What is a blueprint?

A blueprint is a group of related routes. We have 4: `auth` (login/register), `main` (campus, chat, resources, market), `map` (campus map and events), and `study` (study rooms). It keeps code organized instead of having hundreds of routes in one file.

Q: How is the database structured?

We use SQLAlchemy models — each class is a table. The main relationships are: `User` ↔ `Campus` (via

`CampusMember`), `Campus` → `ChatRoom` → `ChatMessage` , `Campus` → `StudyRoom` → `StudyRoomMessage` ,
`Campus` → `Event` / `Resource` / `TutorPost` .

Q: How do you secure passwords?

We use `bcrypt` . When registering, the password is hashed with a random salt:

`bcrypt.hashpw(password.encode(), bcrypt.gensalt())` . The hash is stored, never the password. On login, `bcrypt.checkpw()` compares the entered password to the stored hash.

Q: What is the difference between chat rooms and study rooms?

Chat rooms use **HTTP polling** — the browser asks the server every second for new messages. Study rooms use **Socket.IO (WebSockets)** — a persistent connection where the server pushes messages to clients instantly. WebSockets are more efficient but more complex to set up.

Q: How does email verification work?

We use `itsdangerous.URLSafeTimedSerializer` to generate a signed token containing the user's email. The token is emailed as a link. When clicked, the server decodes and verifies the token (expires in 1 hour, can't be faked without the SECRET_KEY). Only then is `is_verified` set to `True` .

Q: How does the domain restriction on campuses work?

When creating or joining a campus with a domain set (e.g., `usal.edu.lb`), the user must have a verified email ending in that domain. The `has_verified_domain()` method on `User` checks both the primary email and any verified secondary emails.

Q: How does the map work?

The frontend loads **Leaflet.js**, an open-source map library. It fetches pin data from `GET /api/map-data/<campus_id>` which returns JSON with coordinates for all clubs, offices, and active events. Leaflet places markers at those coordinates on an OpenStreetMap base layer.

Q: How do you handle file uploads?

Files are saved to `app/static/uploads/` . For resources (PDFs, etc.), we prefix the filename with a UUID (`uuid.uuid4().hex`) to avoid filename collisions. We store the original filename separately in the database so downloads use the correct name via `send_from_directory(..., download_name=original_filename)` .

Q: How does the notification system work?

`Notification` records are created in the database whenever something happens (new member, new study room joiner, event starting). The badge count in the navbar is updated by `main.js` polling `GET /api/notifications/unread-count` every 30 seconds. The `before_app_request` hook in `main/routes.py` auto-creates notifications for events that have started.

Q: Why no inline scripts or styles?

All CSS is in external `.css` files, all JavaScript in external `.js` files. This keeps templates clean (only structure, no logic), makes code reusable, and follows best practices (separation of concerns, CSP compatibility, easier maintenance).